

Geometry

Index

Setup:

```
typedef complex<double> point;
#define eps 1e-9 // when using heavy trig functions like acos() and
polar(), change it to 1e-6, also consider the problem constraints and
try to make it as forgiving as possible
const double pi = acos(-1.0);
#define X real()
#define Y imag()
#define angle(a) (atan2((a).imag(), (a).real()))
#define same(p1,p2) (abs(p1 - p2) < eps)
#define normalize(a) (a)/abs(a)
#define rotate0(p,ang) (p * polar(1.0, ang))
//(p)*exp(point(0,ang))
#define rotateA(p,ang,about) (rotate0((p - about),ang)+about)
#define reflect0(v,m) (conj((v)/(m))*(m)) //reflect v about m
int dcmp(double a, double b) { return (fabs(a-b) <= eps ? 0 : a < b ?
-1 : 1); }
```

Translation:

```
point translate(point v, point p) {return p+v;}
```

Scaling:

```
point scale(point a, point b, double factor) {
    return a + (b-a)*factor;
}
```

Reflection:

Reflect point c around line AB

```
point reflect(point a, point b, point c) {
    point z = c-a, w = b-a;
    return conj(z/w)*w + a;
}
```

Dot product:

```
double dot(point v, point u) {  
    return (conj(v)*u).X;  
}
```

Cross product:

```
double cross(point v, point u) {  
    return (conj(v)*u).Y;  
}
```

Linear transformation:

Given points p, q, r, and points fp, fq which are points p and q after applying the transformation to them.

Determine point r after transformation.

```
point linearTransform(point p, point q, point r, point fp, point fq)  
{  
    point pq = q-p, num{cross(pq, fq-fp), dot(pq, fq-fp)};  
    return fp + point{cross(r-p, num), dot(r-p, num)} / norm(pq, pq);  
}
```

Are two vectors perpendicular:

```
bool isPerp(point v, point w) {return dot(v,w) == 0;}
```

Orientation:

Positive if c is to the left of AB, negative if c is to the right, 0 if they are collinear

```
double orient(point a, point b, point c) {return cross(b-a,c-a);}
```

Check if point p lies between two lines AB and AC

```
bool inAngle(point a, point b, point c, point p) {  
    if (orient(a,b,c) < 0) swap(b,c);  
    return orient(a,b,p) >= 0 && orient(a,c,p) <= 0;  
}
```

Angle between two vectors:

The smaller angle between two vectors:

```
double unsignedAngle(point v, point w) {  
    return acos(clamp(dot(v,w) / abs(v) / abs(w), -1.0, 1.0));  
}
```

Directed angle using cross and dot product

```
double signedAngle(point v, point w) {  
    return atan2(cross(v, w), dot(v, w));  
}
```

Directed angle using orient function

The angle BAC, going from BA to CA in counterclock wise direction:

```
double directAngle(point a, point b, point c) {  
    if (orient(a,b,c) >= 0)  
        return angle(b-a, c-a);  
    else  
        return 2*pi - angle(b-a, c-a);  
}
```

Is a polygon convex:

Given a set of consecutive vertices, determine if they form a convex polygon.

```
bool isConvex(vector<point> &p) {
    bool hasPos=false, hasNeg=false;
    for (int i=0, n=p.size(); i<n; i++) {
        int o = orient(p[i], p[(i+1)%n], p[(i+2)%n]);
        if (o > 0) hasPos = true;
        if (o < 0) hasNeg = true;
    }
    return !(hasPos && hasNeg);
}
```

Polar sort:

sort vectors by direction. Points are sorted in the order that a rotating ray emanating from the origin would touch them.

```
bool half(point p) { // true if in blue half
    assert(p.X != 0 || p.Y != 0); // the argument of (0,0) is
    undefined
    return p.Y > 0 || (p.Y == 0 && p.X < 0);
}

void polarSort(vector<point> &v) {
    sort(v.begin(), v.end(), [](point v, point w) {
        return make_tuple(half(v), 0) < make_tuple(half(w), cross(v,w));
    });
}
```

points that are in the exact same direction are considered equal, and thus will be sorted arbitrarily. If we want, we can use their magnitude as a tie breaker so that points further from the origin will appear later:

```
void polarSort(vector<point> &v) {
    sort(v.begin(), v.end(), [](point v, point w) {
        return make_tuple(half(v), 0, norm(v,v)) <
        make_tuple(half(w), cross(v,w), norm(w,w));
    });
}
```

Performing polar sort around some point O other than the origin:

```
void polarSortAround(point o, vector<point> &v) {
    sort(v.begin(), v.end(), [](point v, point w) {
        return make_tuple(half(v-o), 0) < make_tuple(half(w-o),
cross(v-o, w-o));
    });
}
```

To set vector v to be the first angle in the polar sort:

```
point v = { /* whatever you want except 0,0 */ };
bool half(point p) {
    return cross(v,p) < 0 || (cross(v,p) == 0 && dot(v,p) < 0);
}
```

Is collinear:

```
bool isCollinear(point a, point b, point c) {
    return fabs( cross(b-a, c-a) ) < eps;
}
```

Is point on a ray:

```
bool isPointOnRay(point a, point b, point c) {
    if(abs(c-a) < eps) return true;
    return same( normalize(b-a), normalize(c-a) );
}
```

Is point on segment:

```
bool isPointOnSegment(point a, point b, point c) {
    if (dcmp(cross(b - a, c - a), 0) != 0) return false;
    return dcmp(dot(a - c, b - c), 0) <= 0;
}
```

Distance to line:

```
double distToLine( point a, point b, point c ) {  
    return fabs(cross(b-a, c-a) / abs(a-b));  
}
```

Distance to segments:

Returns the distance and calculates the point of intersection:

```
double distToSegment(point a, point b, point c, point &intersect) {  
    double d1, d2;  
    point v = b-a, w = c-a;  
    if( (d1 = dot(v, w) ) <= 0 ) {  
        intersect = a;  
        return abs(c-a);  
    }  
    if( (d2 = dot(v, v) ) <= d1 ) {  
        intersect = b;  
        return abs(c-b);  
    }  
    double t = d1/d2;  
    intersect = a + v*t;  
    return abs(c - (a + v*t));  
}
```

Segment intersection:

To use this, segments must be intersecting in one point only, otherwise it returns false.

```
bool intersectSegments(point a, point b, point c, point d, point &
intersect) {
    double d1 = cross(a - b, d - c), d2 = cross(a - c, d - c), d3 =
cross(a - b, a - c);
    if (fabs(d1) < eps)
        return false; // Parallel || identical

    double t1 = d2 / d1, t2 = d3 / d1;
    intersect = a + (b - a) * t1;

    if (t1 < -eps || t2 < -eps || t2 > 1 + eps)
        return false; //e.g ab is ray, cd is segment ... change to
whatever
    return true;
}
```

This one handles all special cases but doesn't calculate the intersection point

```
// Where is P2 relative to segment p0-p1?
// ccw = +1 => angle > 0 or collinear after p1
// cw = -1 => angle < 0 or collinear after p0
// Undefined = 0 => Collinear in range [a, b]. Be careful here
int ccw(point a, point b, point c) {
    point v1(b - a), v2(c - a);
    double t = cross(v1, v2);
    if (t > +eps)
        return +1;
    if (t < -eps)
        return -1;
    if (v1.X * v2.X < -eps || v1.Y * v2.Y < -eps)
        return -1;
    if (norm(v1) < norm(v2) - eps)
        return +1;
    return 0;
}
```

```

bool intersect(point p1, point p2, point p3, point p4) {
    // special case handling if a segment is just a point
    bool x = (p1 == p2), y = (p3==p4);
    if(x && y) return p1 == p3;
    if(x) return ccw(p3, p4, p1) == 0;
    if(y) return ccw(p1, p2, p3) == 0;

    return ccw(p1, p2, p3) * ccw(p1, p2, p4) <= 0 &&
           ccw(p3, p4, p1) * ccw(p3, p4, p2) <= 0;
}

```

Triangle equations:

Sides and angles

```

double fixAngle(double A) {
    return A > 1 ? 1 : (A < -1 ? -1 : A);
}

// sin(A)/a = sin(B)/b = sin(C)/c
double getSide_a_bAB(double b, double A, double B) {
    return (sin(A)*b)/sin(B);
}

double getAngle_A_abB(double a, double b, double B) {
    return asin( fixAngle( (a*sin(B))/b ) );
}

// a^2 = b^2 + c^2 - 2*b*c*cos(A)
double getAngle_A_abc(double a, double b, double c) {
    return acos( fixAngle( (b*b+c*c-a*a)/(2*b*c) ) );
}

```

Triangle area

```
double triangleArea( point p0, point p1, point p2 ) {
    double a = abs(p0-p1), b = abs(p0-p2), c = abs(p1-p2);
    double s = (a+b+c)/2;
    return sqrt((s-a)*(s-b)*(s-c)*s); //Heron's formula
}

// Given the length of three medians of a triangle, find area
double triangleArea( double m1, double m2, double m3 )
{
    // Area of triangle using medians as sides =
    // 3/4 * (area of original triangle)
    if(m1<=0 || m2<=0 || m3<=0 )          return -1; // impossible
    // For area made by sides as medians
    double s = 0.5 * ( m1 + m2 + m3 );
    double medians_area = (s * ( s - m1 ) * (s - m2) * ( s - m3 ));
    double area = 4.0/3.0 * sqrt(medians_area);
    if(medians_area <= 0.0 || area <= 0) return -1; // impossible
    return area;
}
```

Find a circle passing through 3 points:

It returns the radius and the center point, if the points are collinear it returns {-1, (0,0)}

```
pair<double, point> findCircle(point a, point b, point c) {
    point m1 = (b + a) * 0.5;
    point m2 = (b + c) * 0.5;
    point pv1 = (b - a) * point(0.0, 1.0);
    point pv2 = (b - c) * point(0.0, 1.0);
    point center;
    if (!intersectSegments(m1, m1 + pv1, m2, m2 + pv2, center)) {
        return {-1.0, point(0, 0)};
    } // change intersectSegments to line-line intersection
    return {abs(a - center), center};
}
```

Circle-line intersection:

Returns a vector containing the intersection points between line p0p1 and circle

```
vector<point> intersectLineCircle(point p0, point p1, point C, double
r) {
    double a = dot(p1-p0, p1-p0), b = 2*dot(p1-p0, p0-C), c =
dot(p0-C, p0-C) - r*r;
    double f = b*b - 4*a*c;

    vector<point> v;
    if( dcmp(f, 0) >= 0) {
        if( dcmp(f, 0) == 0)    f = 0;
        double t1 =(-b + sqrt(f))/(2*a);
        double t2 =(-b - sqrt(f))/(2*a);
        v.push_back( p0 + t1*(p1-p0) );
        if( dcmp(f, 0) != 0)    v.push_back( p0 + t2*(p1-p0) );
    }
    return v;
}
```

Circle-Circle intersection:

Returns a vector containing the intersection points. 3 if they are identical circles.

```
vector<point> intersectCircleCircle(point c1, double r1, point c2,
double r2) {
    // Handle infinity case first: same center/radius and r > 0
    if (same(c1, c2) && dcmp(r1, r2) == 0 && dcmp(r1, 0) > 0)
        return vector<point>(3, c1);    // infinity 2 same circles
    (not points)

    // Compute 2 intersection case and handle 0, 1, 2 cases
    double ang1 = angle(c2 - c1), ang2 = getAngle_A_abc(r2, r1, abs(c2
- c1));

    if(::isnan(ang2)) // if r1 or d = 0 => nan in getAngle_A_abc (/0)
        ang2 = 0; // fix corruption

    vector<point> v(1, polar(r1, ang1 + ang2) + c1);

    // if point NOT on the 2 circles = no intersection
    if(dcmp(abs(v[0]-c1), r1) != 0 || dcmp(abs(v[0]-c2), r2) != 0 )
        return vector<point>();

    v.push_back(polar(r1, ang1 - ang2) + c1);
    if(same(v[0], v[1])) // if same, then 1 intersection only
        v.pop_back();
    return v;
}
```

To determine the relative positioning of two circles check the relation between r_1 , r_2 , distance between center:

.

Far apart if $r_1 + r_2 < d$

Inside each other but not touching if $r_1 - r_2 > d$

Touching from the inside if $|r_1 - r_2| = d$

Touching from the outside if $r_1 + r_2 = d$

Overlapping in 2 points if $r_1 + r_2 > d > |r_1 - r_2|$

Minimal enclosing circle:

Returns 3 points in array r resembling the minimal circle that encloses some points.

```
const int MAX = 100000+9;
point pnts[MAX], r[3], cen;
double rad;
int ps, rs; // ps = n, rs = 0, initially

// Pre condition
// random_shuffle(pnts, pnts+ps);      rs = 0;
void MEC() {
    if(ps == 0 && rs == 2) {
        cen = (r[0]+r[1])/2.0;
        rad = abs(r[0]-cen);
    }
    else if(rs == 3) {
        pair<double, point> p = findCircle(r[0], r[1], r[2]);
        cen = p.second;
        rad = p.first;
    }
    else if(ps == 0) {
        cen = r[0]; // sometime be garbage, but will not affect
        rad = 0;
    }
    else {
        ps--;
        MEC();

        if(abs(pnts[ps]-cen) > rad) {
            r[rs++] = pnts[ps];
            MEC();
            rs--;
        }

        ps++;
    }
}
```

Is square?

Given 4 points determine if they form a non zero area square.

```
bool isSquare(point a, point b, point c, point d) {
    point v1 = b-a, v2 = d-a, v3 = c-a;
    if(abs(v1) < eps || abs(v2) < eps || abs(v3) < eps) return false;
    if(same((v1+v2), v3) && fabs(dot(v1,v2)) < eps && fabs(abs(v1) -
abs(v2)) < eps) return true;
    if(same((v1+v3), v2) && fabs(dot(v1,v3)) < eps && fabs(abs(v1) -
abs(v3)) < eps) return true;
    if(same((v2+v3), v1) && fabs(dot(v2,v3)) < eps && fabs(abs(v2) -
abs(v3)) < eps) return true;
    return false;
}
```

Is rectangle?

Given 4 points determine if they form a non zero area rectangle.

```
bool isRectangle(point a, point b, point c, point d) {
    point v1 = b - a, v2 = d - a, v3 = c - a;
    if (abs(v1) < eps || abs(v2) < eps || abs(v3) < eps) return false;
    if (same(v1 + v2, v3) && fabs(dot(v1, v2)) < eps) return true;
    if (same(v1 + v3, v2) && fabs(dot(v1, v3)) < eps) return true;
    if (same(v2 + v3, v1) && fabs(dot(v2, v3)) < eps) return true;
    return false;
}
```

Is parallelogram?

Given 4 points determine if they form a non zero area parallelogram.

```
bool isParallelogram(point a, point b, point c, point d) {
    point v1 = b - a, v2 = d - a, v3 = c - a;
    if (abs(v1) < eps || abs(v2) < eps || abs(v3) < eps) return false;
    if (same(v1 + v2, v3) && fabs(cross(v1, v2)) > eps) return true;
    if (same(v1 + v3, v2) && fabs(cross(v1, v3)) > eps) return true;
    if (same(v2 + v3, v1) && fabs(cross(v2, v3)) > eps) return true;
    return false;
}
```

Is simple polygon?

```
// assume p0 is added again to the end of vector p (n+1 points)
bool isSimplePolygon(vector<point> &p) {
    for (int i = 0; i < sz(p) - 1; i++) {
        for (int j = i + 2; j < sz(p) - 1; j++)
            if (intersect(p[i], p[i + 1], p[j], p[j + 1]) && (i != 0 || j
!= sz(p) - 2)) /* last/first edges are consecutive*/
                return false;    //Segments must not share vertices
    }
    return true;
}
```

Is convex polygon, using ccw

```
bool isConvexPolygon(vector<point> &p) {  
    // all polygon 3 consecutive points must have same sign (ccw  
    or cw)  
    p.push_back(p[0]), p.push_back(p[1]); // wrap points for  
    simplicity  
  
    int sign = ccw(p[0], p[1], p[2]);  
    bool ok = true;  
    for (int i = 1; ok && i < sz(p) - 2; i++) {  
        if (ccw(p[i], p[i + 1], p[i + 2]) != sign)  
            ok = false;  
    }  
    p.pop_back(), p.pop_back();  
    return ok;  
}  
// isConcave = !isConvex  
// isConcave = 2 different signs
```

Polygon area

```
double polygonArea(vector<point>& points) {  
    long double a = 0;  
    for(int i = 0; i < points.size(); i++) a += cross(points[i],  
    points[ (i+1) % sz(points)]);  
    return fabs(a/2.0);    // If area > 0 then points ordered ccw  
}
```

Polygon centroid

```
point polygonCenteriod(vector<point> points) {
    double x = 0, y = 0, a = 0, c;
    for(int i = 0; i < points.size(); ++i)
    {
        int j = (i + 1) % sz(points);
        c = cross(points[i], points[j]), a += c;
        x += (points[i].X + points[j].X) * c;
        y += (points[i].Y + points[j].Y) * c;
    }
    if (dcmp(a, 0) == 0)
        return (points[0] + points.back()) * 0.5;    // Line
    a /= 2, x /= 6 * a, y /= 6 * a;
    // Fix values in case
    if (dcmp(x, 0) == 0)    x = 0;
    if (dcmp(y, 0) == 0)    y = 0;
    return point(x, y);
}
```

Convex polygon cut

```
// P needs to be counterclockwise convex polygon
// if P is sorted clockwise swap left and right
pair<vector<point>, vector<point>> polygonCut(vector<point> &p, point
A, point B) {
    vector<point> left, right;
    point intersect;
    for (int i = 0; i < sz(p); ++i) {
        point cur = p[i], nxt = p[(i + 1) % sz(p)];
        if (cross(B-A, cur-A) >= 0)
            right.push_back(cur);
        //NOTE adjust intersectSegments to handle AB as line
        if (intersectSegments(A, B, cur, nxt, intersect)) {
            right.push_back(intersect);
            left.push_back(intersect);
        }
        if (cross(B-A, cur-A) <= 0)
```

```

        left.push_back(cur);
    }
    return make_pair(left, right);
}

```

Is a point inside a polygon?

Works on simple and non-simple polygons.

```

bool isInsidePoly(vector<point>& p, point p0) {
    int wn = 0; // the winding number counter

    for (int i = 0; i < sz(p); i++) {
        point cur = p[i], nxt = p[(i + 1) % sz(p)];
        if (isPointOnSegment(cur, nxt, p0))
            return true;
        if (cur.Y <= p0.Y) { // Upward edge
            if (nxt.Y > p0.Y && cross(nxt-cur, p0-cur) > eps)
                ++wn;
        } else { // Downward edge
            if (nxt.Y <= p0.Y && cross(nxt-cur, p0-cur) < -eps)
                --wn;
        }
    }
    return wn != 0;
}

```

Is point inside a convex polygon $O(\log n)$

Returns true if point q is on the polygon

```
// Points must be sorted in ccw
bool isInsideConvexPolygon(const vector<point>& poly, point q) {
    int n = poly.size();
    if (n < 3) return false;
    if (dcmp(orient(poly[0], poly[1], q), 0) < 0) {
        return false;
    }
    if (dcmp(orient(poly[0], poly[n - 1], q), 0) > 0) {
        return false;
    }
    int low = 1;
    int high = n - 1;
    while (high - low > 1) {
        int mid = low + (high - low) / 2;
        if (dcmp(orient(poly[0], poly[mid], q), 0) >= 0) {
            low = mid;
        } else {
            high = mid;
        }
    }
    if (dcmp(orient(poly[low], poly[high], q), 0) >= 0) {
        return true;
    }

    return false;
}
```

Is point strictly inside a convex polygon?

```
// points must be sorted in ccw
bool isStrictlyInsideConvexPolygon(const vector<point>& poly, point
q) {
    int n = poly.size();
    if (n < 3) return false;
    if (dcmp(orient(poly[0], poly[1], q), 0) <= 0) {
        return false;
    }
    if (dcmp(orient(poly[0], poly[n - 1], q), 0) >= 0) {
        return false;
    }
    int low = 1;
    int high = n - 1;
    while (high - low > 1) {
        int mid = low + (high - low) / 2;
        if (dcmp(orient(poly[0], poly[mid], q), 0) >= 0) {
            low = mid;
        } else {
            high = mid;
        }
    }
    if (dcmp(orient(poly[low], poly[high], q), 0) > 0) {
        return true;
    }

    return false;
}
```

Sort polygon in ccw order

```
void orderPolygonCCW(vector<point>& poly) {  
    if (poly.size() < 3) return;  
    double sumX = 0, sumY = 0;  
    for (point p : poly) {  
        sumX += p.X;  
        sumY += p.Y;  
    }  
    point center(sumX / poly.size(), sumY / poly.size());  
    polarSortAround(center, poly);  
}
```